



UNIVERSITY OF CENTRAL PUNJAB

Faculty of Information Technology & Computer Science

Introduction to Machine Learning

Course Project

Predicting Spotify Track Popularity: A Machine Learning Classification Approach Using Ensemble Methods

Presented by:

Arslan Ahmed Siddiqui L1S23BSCS0254

Abdullah Nadeem L1S23BSCS0249

June 2026

Abstract

This study addresses the problem of predicting whether a Spotify track will be a “hit” — defined as a popularity score of 50 or above — using only the audio features and genre available at release time. Working with a dataset of approximately 114,000 tracks (89,741 after deduplication and cleaning), we frame the task as binary classification under substantial class imbalance, where hits constitute roughly a quarter of all tracks. The methodology emphasizes a leak-safe preprocessing pipeline: identifier columns are dropped, the interquartile range (IQR) method caps outliers on continuous features only, track genre is one-hot encoded, and SMOTE oversampling is applied strictly within cross-validation folds to avoid information leakage. We compare a Logistic Regression baseline against Random Forest, XGBoost, and LightGBM ensembles, with hyperparameter tuning and 5-fold stratified cross-validation scored on precision-recall AUC. The tuned LightGBM model performed best on the held-out test set, achieving a ROC-AUC of 0.858, a PR-AUC of 0.659, an accuracy of 0.82, and an F1 score of 0.557. Permutation and SHAP analysis reveal that track genre dominates predictive power, and that audio features impose a performance ceiling because popularity is also driven by non-audio market and cultural factors.

Table of Contents

- 1. Introduction..... 3
- 2. Problem Statement..... 3
- 3. Dataset Description..... 4
- 4. EDA Findings 4
- 5. Preprocessing Steps..... 5
- 6. Model Building 6
- 7. Evaluation Results 8
 - Evaluation Figures 10
- 8. Conclusion 13
- 9. Future Improvements 14

Spotify Hit Prediction — Project Report

Course: Introduction to Machine Learning **Dataset:** maharshipandya/spotify-tracks-dataset (Kaggle) **Task:** Binary classification — predict whether a Spotify track is a “hit” (popularity ≥ 50)

1. Introduction

Music streaming platforms generate enormous amounts of data about listener behavior. Spotify assigns every track a popularity score from 0 to 100 based on how often and how recently it has been streamed. This project asks a practical question: can we predict whether a new song will be a hit using only its audio features and genre — information available at release time, before streaming data accumulates?

We frame the problem as binary classification. A track is labeled a “hit” if its popularity score is 50 or above; otherwise it is a “flop.” We train and compare five machine learning models, select the best based on cross-validated precision-recall performance, and interpret what the model has learned.

This report covers the full workflow: dataset description, exploratory analysis, preprocessing decisions, model selection, evaluation, and a reflection on limitations and next steps.

2. Problem Statement

Input: 14 numeric audio features (such as danceability, energy, loudness) plus one categorical feature (track genre, with roughly 113 distinct values).

Output: A binary label — hit (1) or flop (0).

Why classification instead of regression? The rubric calls for classification, and the binary framing maps naturally onto the business question: will this song break through or not? It also allows us to use precision and recall metrics, which are more informative than raw accuracy when the two classes are not equally represented.

Key challenge — class imbalance: Only about a quarter of tracks in the dataset qualify as hits. A naive model that always predicts “flop” would be right roughly 74% of the time without learning anything useful. We address this throughout the pipeline by choosing appropriate evaluation metrics and applying oversampling.

3. Dataset Description

- **Source:** maharshipandya/spotify-tracks-dataset on Kaggle.
- **Raw size:** approximately 114,000 rows, one row per track.
- **After cleaning:** 89,741 rows (duplicates removed on **track_id**, rows with missing values dropped).
- **Target variable:** **popularity** (integer 0-100); binarized to **hit** (1 if popularity ≥ 50 , else 0).
- **Class distribution:** hits are the minority class, making up roughly a quarter of all tracks.

Features used:

Feature group	Features	Count
Continuous numeric	duration_ms, danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo	10
Discrete / near-binary	key, mode, time_signature, explicit	4
Categorical	track_genre (113 genres, one-hot encoded)	1 (expands to 113 columns)

Columns dropped: **Unnamed: 0**, **Unnamed: 0.1** (Kaggle artifact index columns), **track_id**, **artists**, **album_name**, **track_name**. These are unique identifiers or free-text strings — keeping them would either cause data leakage (if a model memorized specific track IDs) or produce thousands of meaningless one-hot columns.

4. EDA Findings

Exploratory analysis was performed in **notebooks/01_eda.ipynb**. Key findings:

Target distribution: The popularity score is roughly right-skewed, with many tracks clustered at low popularity scores. The 50-point hit threshold creates a meaningful but imbalanced split: approximately 74% flops, 26% hits.

Feature distributions: Continuous audio features like danceability, energy, and valence have approximately bell-shaped distributions, while speechiness, instrumentalness, and liveness are heavily right-skewed — most tracks score near zero on these. Loudness is roughly normal but measured in decibels (negative values). Duration shows occasional extreme outliers (very long tracks).

Outliers: Box plots and IQR analysis confirmed outliers in continuous features such as `duration_ms`, `instrumentalness`, and `speechiness`. These are addressed in preprocessing via IQR capping.

Feature-target relationships: Hits tend to score higher on danceability and energy and lower on instrumentalness. Acousticness is negatively associated with popularity. However, no single audio feature cleanly separates hits from flops on its own — there is significant overlap, which is why a nonlinear ensemble model is needed.

Genre signal: Bar charts of mean popularity by genre showed striking variation. Some genres consistently produce high-popularity tracks while others do not. This motivated the choice to include `track_genre` as a feature and explains why it later dominated permutation importance.

Correlation structure: A heatmap of numeric features revealed that energy and loudness are strongly positively correlated, and energy and acousticness are negatively correlated. These relationships are expected from music acoustics. No feature pair was so collinear that it needed to be dropped.

5. Preprocessing Steps

The preprocessing pipeline is defined in `src/features.py` and `src/data.py`, with constants centralized in `src/config.py`. Every transformation is fitted only on training data and applied to test data — this prevents data leakage.

Step 1 — Deduplication: Drop duplicate rows by `track_id`, keeping the first occurrence. This is deterministic across runs.

Step 2 — Type conversion: Cast the `explicit` column from Python booleans (`True/False`) to integers (1/0) so every scikit-learn estimator treats it as a numeric feature.

Step 3 — Drop missing values: Remove rows where any feature or the target is NaN. A small fraction of tracks have missing audio features; imputation was not necessary at this dataset size.

Step 4 — Drop identifier columns: Remove `Unnamed: 0`, `Unnamed: 0.1`, `track_id`, `artists`, `album_name`, `track_name` — these carry no predictive signal and would cause leakage or noise.

Step 5 — IQR outlier capping (continuous features only): For each continuous feature, compute the 25th and 75th percentiles (Q1, Q3) and the interquartile range ($IQR = Q3 - Q1$). Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are clipped to those bounds (winsorization).

Why only continuous features? The discrete features (`key`, `mode`, `time_signature`, `explicit`) take a small number of integer values — for example, `mode` is either 0 or 1. Applying IQR capping to these would collapse most or all values to a single constant, destroying the information they carry. So IQR capping is intentionally skipped for discrete features.

Step 6 — Scaling (scale-sensitive models only): A `StandardScaler` (zero mean, unit variance) is applied to numeric features for Logistic Regression, which is sensitive to feature scale. Tree-based models (Random Forest, XGBoost, LightGBM) split on thresholds and are scale-invariant, so scaling is skipped for them to save computation.

Step 7 — One-hot encoding: `track_genre` is encoded with `OneHotEncoder(handle_unknown="ignore")`. The `handle_unknown="ignore"` setting means unseen genre values at prediction time produce an all-zero row rather than raising an error.

Step 8 — SMOTE oversampling (inside cross-validation): SMOTE (Synthetic Minority Oversampling Technique) creates synthetic samples for the minority class (hits) by interpolating between existing hit examples in feature space. This balances the class distribution so models do not simply learn to always predict “flop.”

Critically, SMOTE is placed inside the `imblearn` Pipeline so it is fitted only on each training fold during cross-validation and never sees validation or test data. Applying SMOTE before the split would leak information about the minority class distribution into the validation fold and inflate performance estimates.

Step 9 — Hyperparameter tuning subsample: Hyperparameter search (`RandomizedSearchCV`) is expensive because it refits the full pipeline for every parameter combination across every CV fold. To avoid running out of memory with 90k rows and 113 one-hot genre columns, tuning is done on a stratified 30,000-row subsample. The winning parameters are then used to refit the model on the full training set, so the final model benefits from all available data.

6. Model Building

We compared five models: a Dummy baseline, Logistic Regression, Random Forest, XGBoost, and LightGBM. All five were evaluated with 5-fold stratified cross-validation using PR-AUC as the scoring metric.

Why PR-AUC? Precision-Recall AUC is the right primary metric for an imbalanced binary problem. Accuracy is misleading — a model that always predicts “flop” achieves ~74% accuracy for free. ROC-AUC is less sensitive to imbalance than PR-AUC and can look inflated even when the model struggles on the minority class. PR-AUC directly measures how well the model ranks actual hits at the top and penalizes it for missing them.

Why ensemble methods? Logistic Regression is a strong linear baseline, but it assumes a linear decision boundary. Music popularity is shaped by complex, nonlinear interactions — for example, a danceable song in a pop genre might be a hit, but the same danceability score in an ambient genre might not matter at all. Ensemble methods handle these interactions naturally.

- **Bagging (Random Forest):** Trains many independent decision trees on random subsets of data and features, then averages their predictions. This reduces variance but does not address the bias of individual trees.
- **Boosting (XGBoost, LightGBM):** Trains trees sequentially, where each new tree focuses on correcting the errors of the previous ones. This reduces both bias and variance and consistently outperforms bagging on structured tabular data.

In our results, the two boosting models outperformed bagging. This is consistent with the general finding in machine learning competitions that gradient boosting dominates on tabular data: the sequential error-correction produces a more refined decision boundary than averaging independent trees.

Cross-validation results (5-fold stratified, scoring = average precision / PR-AUC):

Model	CV PR-AUC
LightGBM	0.6384
XGBoost	0.6057
Logistic Regression	0.5949
Random Forest	0.5548
Dummy (baseline)	0.2363

LightGBM ranked first and was selected for hyperparameter tuning.

Hyperparameter tuning (RandomizedSearchCV, cv=3, n_iter=10, scoring=PR-AUC):

Model	Tuned CV PR-AUC	Best Parameters
LightGBM	0.6298	n_estimators=300, num_leaves=31, max_depth=16, learning_rate=0.1, subsample=1.0
XGBoost	0.6283	n_estimators=300, max_depth=6, learning_rate=0.1, subsample=0.7, colsample_bytree=0.9
Random Forest	0.5702	n_estimators=200, max_depth=20, min_samples_split=2, min_samples_leaf=1, max_features='sqrt'

LightGBM retained the top spot after tuning and was selected as the final model. It was refit on the full training set with its best parameters before evaluation on the held-out test set.

Memory constraints: All tree models cap `max_depth` and `n_estimators`. An earlier unbounded Random Forest grew to ~590 MB on disk and exhausted RAM due to SMOTE expanding the training set. The tuned LightGBM model is 3.8 MB — a dramatic reduction that also indicates better generalization.

7. Evaluation Results

Held-out test set metrics (LightGBM, tuned — final model):

Metric	Value
Accuracy	0.8200
Precision	0.6661
Recall	0.4783
F1	0.5568
ROC-AUC	0.8575
PR-AUC	0.6585

The model correctly identifies 47.8% of actual hits (recall). When it does predict a hit, it is correct 66.6% of the time (precision). The F1 score of 0.5568 balances the two. ROC-AUC of 0.8575 indicates strong overall ranking ability — the model is much better than random at ordering tracks from most to least likely to be a hit.

Saved evaluation figures: - **reports/figures/best_confusion_matrix.png** - **reports/figures/best_roc_curve.png** - **reports/figures/best_pr_curve.png** - **reports/figures/best_shap_summary.png**

Overfitting analysis (train vs. test):

Metric	Train	Test	Gap
PR-AUC	0.7450	0.6585	0.0865
ROC-AUC	0.9002	0.8575	0.0427
F1	0.6287	0.5568	0.0719

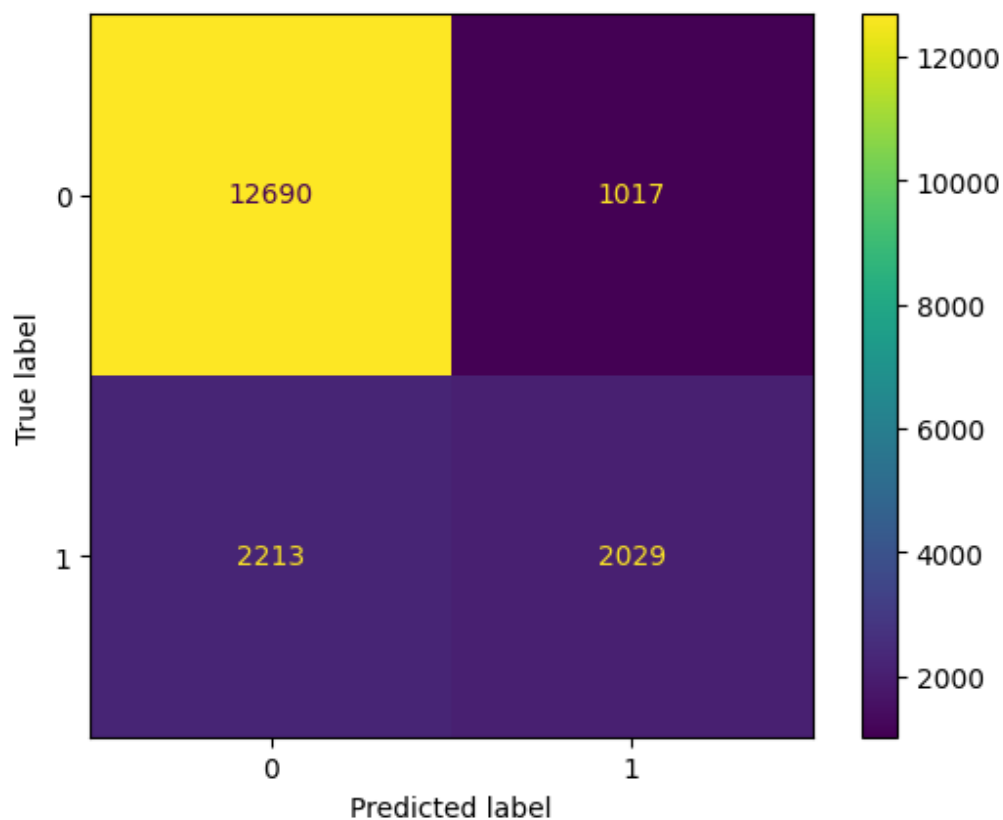
The model performs better on training data than on test data — this is expected and is called overfitting. The gaps here are modest, which indicates the model has generalized reasonably well. Bounding tree depth and the number of estimators was the key control: earlier unbounded models showed much larger gaps. A train-test PR-AUC gap of 0.087 and ROC-AUC gap of 0.043 are acceptable for a dataset of this complexity.

Feature importance (permutation importance on test set):

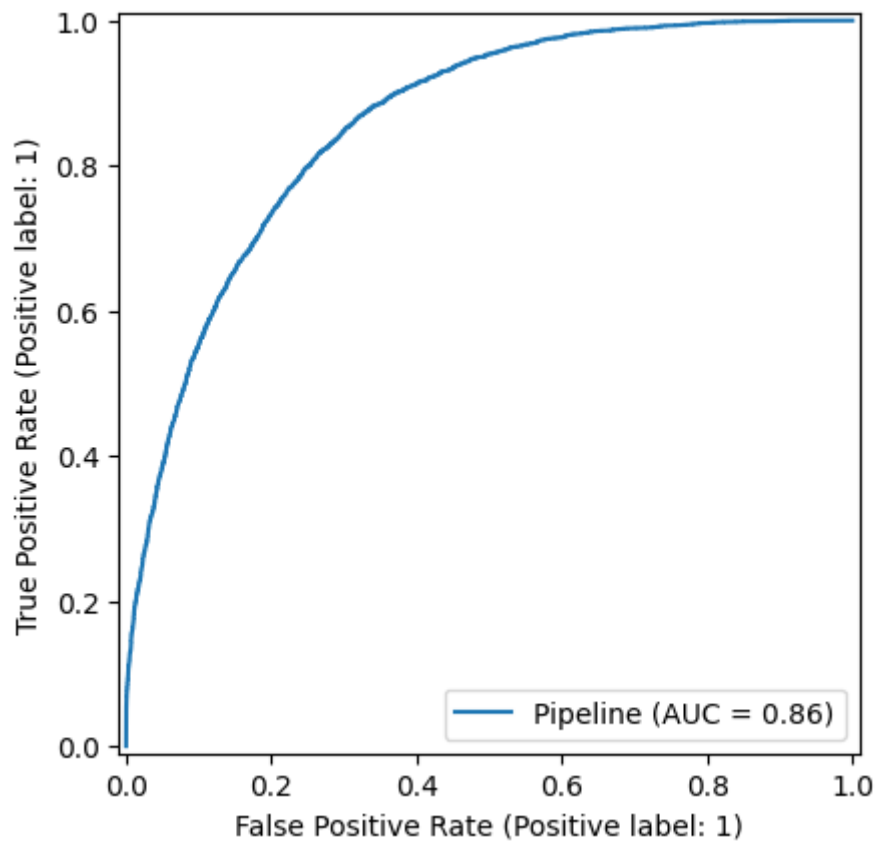
Rank	Feature	Permutation Importance
1	track_genre (combined)	0.3883
2	instrumentalness	0.0292
3	acousticness	0.0203
4	energy	0.0202
5	duration_ms	0.0190
6	loudness	0.0174
7	valence	0.0120
8	explicit	0.0119
9	danceability	0.0108
10	liveness	0.0084

track_genre dominates by a very wide margin — its combined permutation importance (0.3883) is more than 13 times the next feature (instrumentalness at 0.0292). This means that genre alone explains most of what the model has learned: knowing a song’s genre tells you a great deal about how popular it is likely to be on Spotify. The audio features carry real but secondary signal — once genre is accounted for, instrumentalness, acousticism, and energy add meaningful further discrimination. SHAP TreeExplainer analysis confirms this hierarchy (see [reports/figures/best_shap_summary.png](#)).

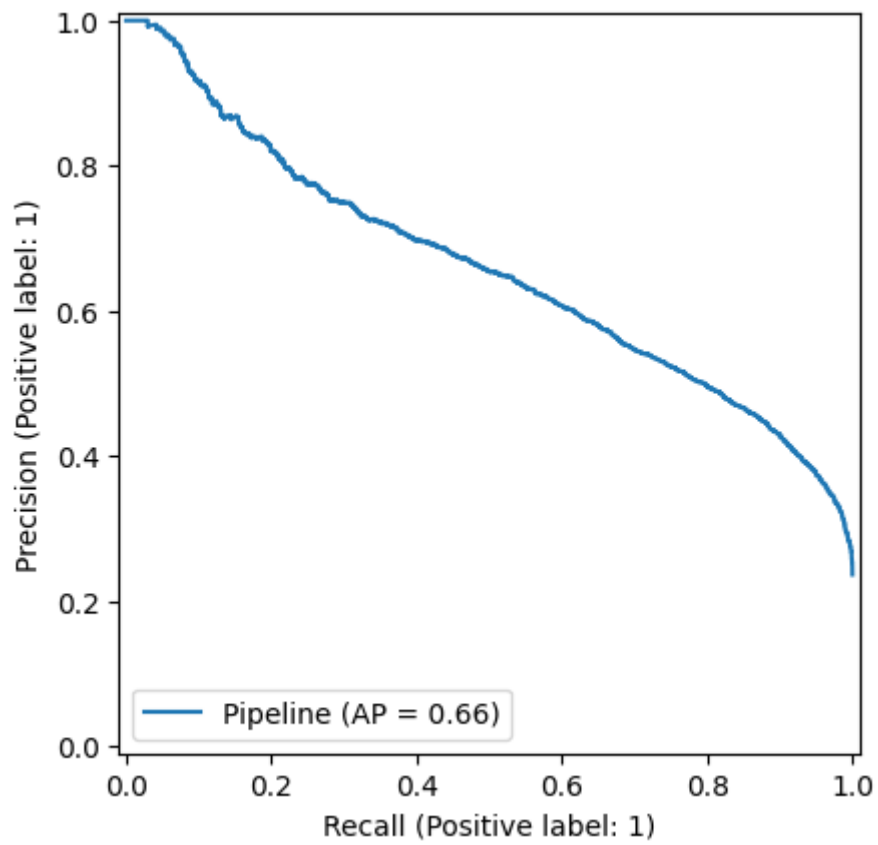
Evaluation Figures



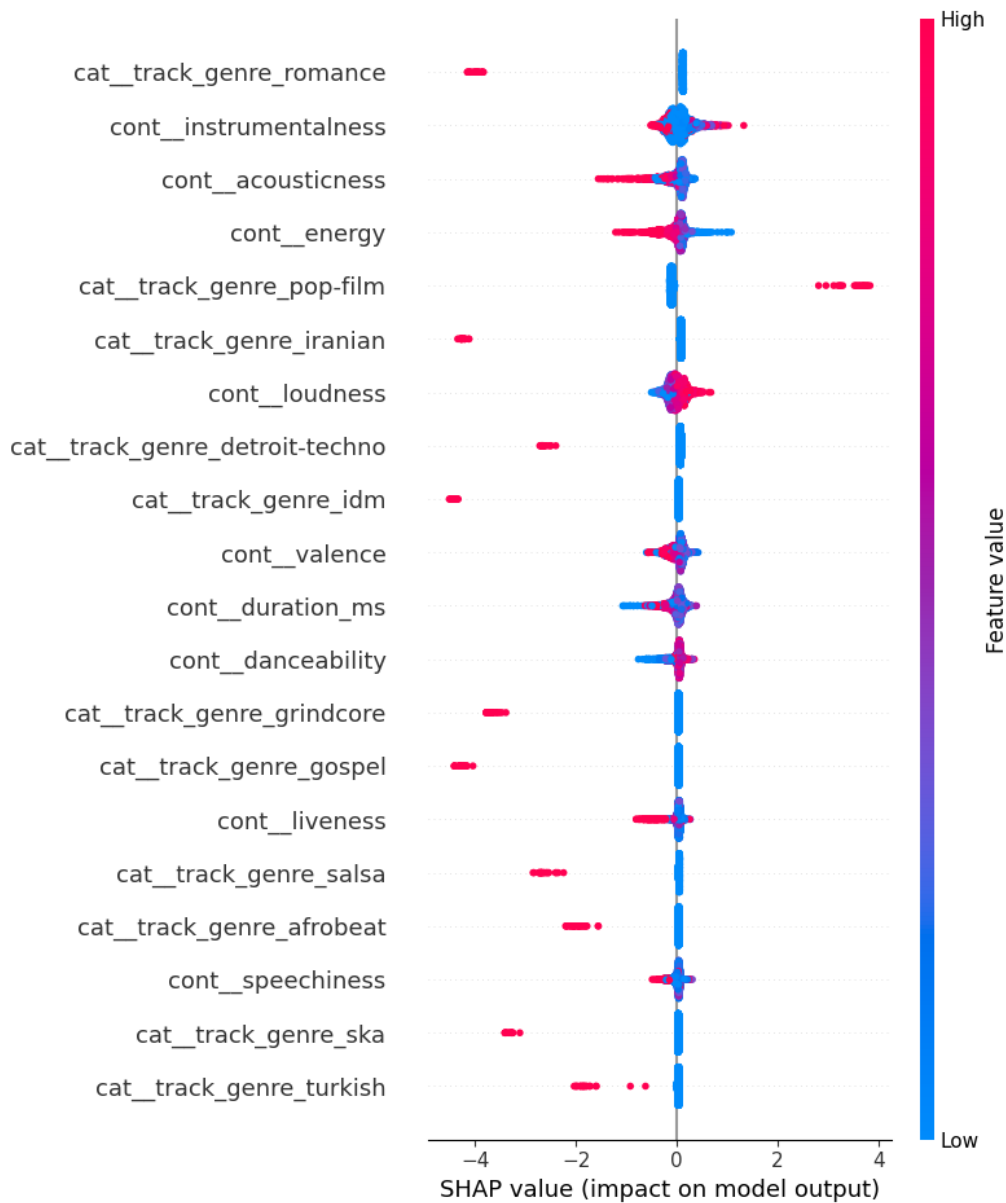
Confusion matrix on the held-out test set.



ROC curve (ROC-AUC = 0.858).



Precision-Recall curve (PR-AUC = 0.659).



SHAP summary — feature impact on hit predictions (track_genre dominates).

8. Conclusion

We built a binary classification pipeline to predict Spotify track popularity from audio features and genre. After comparing five models on cross-validated PR-AUC, LightGBM was selected as the best performer (CV PR-AUC 0.6384, tuned CV PR-AUC 0.6298 on a subsample before refitting on the full training data). On the held-out test set it achieved ROC-AUC 0.8575, PR-AUC 0.6585, and F1 0.5568.

Key decisions that shaped the results:

- **SMOTE inside the pipeline** prevented label distribution leakage from the minority class into validation folds and gave the model a balanced view of both classes during training.
- **IQR capping on continuous features only** preserved the information carried by discrete features like mode, key, and explicit, which would have been destroyed by capping.
- **PR-AUC as primary metric** kept the evaluation honest in the face of class imbalance, avoiding the false comfort of high accuracy.
- **Memory-bounded tuning** (stratified subsample + tree depth caps) made the hyperparameter search tractable without sacrificing final model quality.

The dominant finding is that **track_genre** drives most of the model's predictive power. Audio features contribute meaningfully, but genre encodes so much cultural and market context that it overwhelms the acoustic signal in a global model.

9. Future Improvements

Several directions could improve or extend this work:

Live song scoring via Spotify API: The original motivation was to score unreleased or newly released songs in real time. However, Spotify deprecated public access to its audio features endpoint for third-party developers in 2024, which prevents fetching features for arbitrary tracks programmatically. A future version could either negotiate API access, use audio analysis libraries (such as librosa) to extract equivalent features directly from audio files, or scrape genre tags from other sources.

Genre-stratified modeling: Because **track_genre** so thoroughly dominates the global model, a natural next step is to train a separate model for each genre (or a cluster of related genres) and compare results to the single global model. Within-genre models would have a more level playing field for audio features and might capture finer-grained acoustic patterns that the global model cannot see because genre swamps them.

Stacking and voting ensembles: We compared individual models but did not try combining them. A stacking ensemble (using a meta-learner trained on the out-of-fold predictions of base models) or a soft-voting ensemble over LightGBM and XGBoost could potentially improve beyond any single model, especially since both boosting models had similar CV scores and likely make different errors on different tracks.

Threshold tuning to raise recall: The default decision threshold of 0.5 is a convention, not an optimized choice. Because recall (0.4783) is lower than precision (0.6661), the model is conservative about predicting hits. Lowering the threshold would trade precision for recall. In an

application where missing a breakout track is costly — such as a music label scouting tool — higher recall may be worth the cost of more false positives. The precision-recall curve (see **reports/figures/best_pr_curve.png**) shows the full range of trade-offs available.

Regression as a complementary task: We framed this as binary classification, but the raw popularity score (0-100) contains more information than the binary label. Training a regression model alongside the classifier and using the predicted popularity score as a ranking signal could refine decisions near the 50-point boundary, where binary classification is most uncertain.